



Compiladores

Aula 7: Análise Sintática Descendente

Prof. Alex Torquato Souza Carneiro
alex.carneiro@impacta.edu.br

Agenda

- Parser recursivo
- First / Follow
- Gramáticas LL(1)

Parser recursivo

- A gramática (GLC) não é apenas uma especificação, mas orienta um procedimento executável.
- **Partimos de uma GLC:** produções, derivações e estrutura sintática.
- Em seguida **eliminamos obstáculos à predição:** recursão à esquerda e prefixos comuns.
- Por fim, o **parser descendente** é capaz de reconhecer a entrada e construir a AST.

Objetivo: escolher a produção correta de acordo com o próximo token.

Parser recursivo

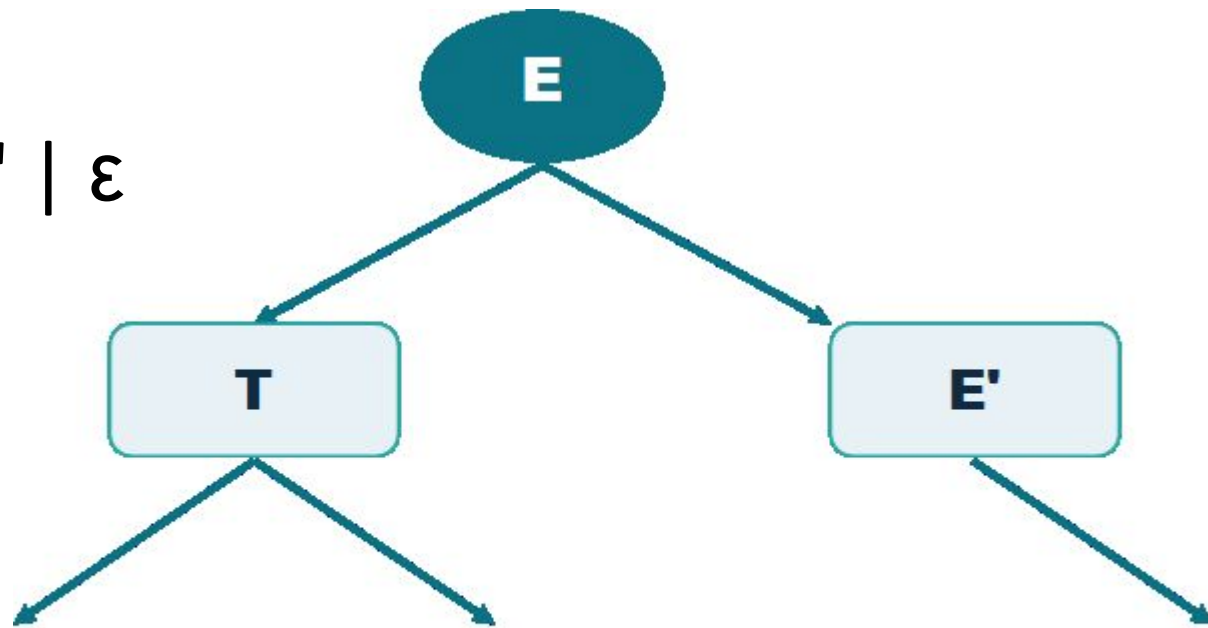
- A ideia do parser é dispormos de uma ferramenta capaz de construir a árvore de derivação de cima para baixo, começando no símbolo inicial.
- O parser expande não-terminais até alcançar os tokens da entrada. A estratégia reproduz uma derivação mais à esquerda.

Parser recursivo

- Exemplo:

$$E \rightarrow T E'$$

$$E' \rightarrow + T E' \mid \varepsilon$$



árvore parcial

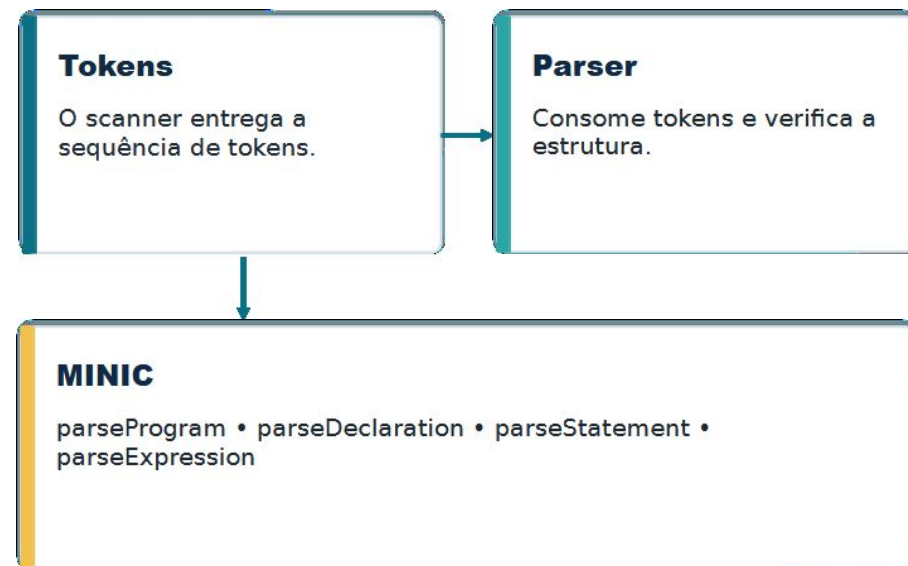
Parser recursivo

- Descida recursiva:
 - cada não-terminal da gramática corresponde a um procedimento do parser.

- Lógica:

```

procedure A() :
  escolher A → X1 X2 ... Xn
  para cada Xi:
    se Xi é não-terminal:
      chamar Xi()
    senão:
      casar Xi com lookahead
  se falhar: sinalizar erro
    
```



Parser recursivo

- Recursão à esquerda:
 - Uma chamada recursiva acontece antes de qualquer consumo de token.
- Produção:

$$E \rightarrow E + T \mid T$$

Transformação:

$$A \rightarrow A \alpha \mid \beta$$

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \varepsilon$$

Resultado:

$$E \rightarrow T E'$$

$$E' \rightarrow + T E' \mid \varepsilon$$

A forma da derivação muda para favorecer a análise descendente.

Parser recursivo

- Fatoração à esquerda:
 - Adiar uma decisão quando alternativas começam pelo mesmo prefixo.

```
stmt → if expr then stmt else stmt
      | if expr then stmt
```



```
stmt → if expr then stmt stmt
stmt' → else stmt | ε
```

Depois de ler if, ainda não é possível decidir.

O parser consome o prefixo comum e decide com um token posterior.

A fatoração não resolve sozinha a ambiguidade do “else vazio”.
É preciso definir uma regra de associação.

FIRST e FOLLOW

- Conjuntos que transformam a gramática em decisões baseadas no lookahead.
- FIRST(α):
 - Terminais que podem iniciar alguma cadeia derivada de α . Se α puder gerar ϵ , então $\epsilon \in \text{FIRST}(\alpha)$.
- FOLLOW(A):
 - Terminais que podem aparecer imediatamente depois de A. Para o símbolo inicial, $\$ \in \text{FOLLOW}$.

FIRST e FOLLOW

- Exemplo:

$$E \rightarrow T E'$$

$$E' \rightarrow + T E' \mid \varepsilon$$

$$T \rightarrow F T'$$

$$T' \rightarrow * F T' \mid \varepsilon$$

$$F \rightarrow (E) \mid \text{id}$$

Se o lookahead é “+”, E' pode expandir para $+ T E'$. Se é “)”, E' pode desaparecer por ε .

O símbolo \$ representa o fim da entrada (end-of-input marker).

Não-terminal	FIRST	FOLLOW
E	{ (, id }	{), \$ }
E'	{ +, ε }	{), \$ }
T	{ (, id }	{ +,), \$ }
T'	{ *, ε }	{ +,), \$ }
F	{ (, id }	{ *, +,), \$ }

FIRST orienta a entrada; FOLLOW orienta a saída por ε .

FIRST e FOLLOW

- O \$ não faz parte da linguagem nem é um token digitado pelo programador. É um marcador usado pelo analisador sintático para indicar que todos os tokens da entrada já foram lidos.
- Por exemplo, se o programa gerar a sequência:

ID + INT

- o parser considera a entrada como:

ID + INT \$

Gramáticas LL(1)

- Os princípios das gramáticas LL(1) incluem:
 - leitura da entrada da esquerda para a direita,
 - derivação mais à esquerda e
 - um token de lookahead.



Left-to-right

entrada da esquerda para a direita



Leftmost

derivação mais à esquerda



One lookahead

um token para decidir

$M[F, id] = F \rightarrow id$

Diagnóstico:

Duas produções na mesma entrada = conflito na tabela; a gramática não está adequada para LL(1).

Gramáticas LL(1)

- A produção $M[F, id] = F \rightarrow id$ é uma entrada da tabela preditiva LL(1).
- Quando o topo da pilha for o não terminal F e o próximo token da entrada for id , o parser deve aplicar a produção $F \rightarrow id$.
 - M : a tabela de análise preditiva;
 - F : a linha da tabela, correspondente ao não terminal que será expandido;
 - id : a coluna da tabela, correspondente ao próximo token de entrada;
 - $F \rightarrow id$: a produção que o parser deve utilizar.

Gramáticas LL(1)

- Como preencher a tabela preditiva.
- A tabela $M[A, a]$ registra a escolha para cada não-terminal e lookahead.

1

FIRST(α)

Para cada terminal $a \in \text{FIRST}(\alpha)$, inserir $A \rightarrow \alpha$.

2

ϵ

Se $\epsilon \in \text{FIRST}(\alpha)$, usar $\text{FOLLOW}(A)$.

3

Conflito

Duas produções na mesma célula indicam problema.

Gramáticas LL(1)

- Parser preditivo com pilha.
- Torna a gramática um mecanismo executável, sem chamadas recursivas explícitas.

Pilha	Entrada
\$ E	id + id \$
\$ T E'	id + id \$
\$ F T' E'	id + id \$
\$ id T' E'	id + id \$
\$ T' E'	+ id \$
\$ + T E'	+ id \$
\$ T E'	id \$

Terminal = token
desempilha e avança

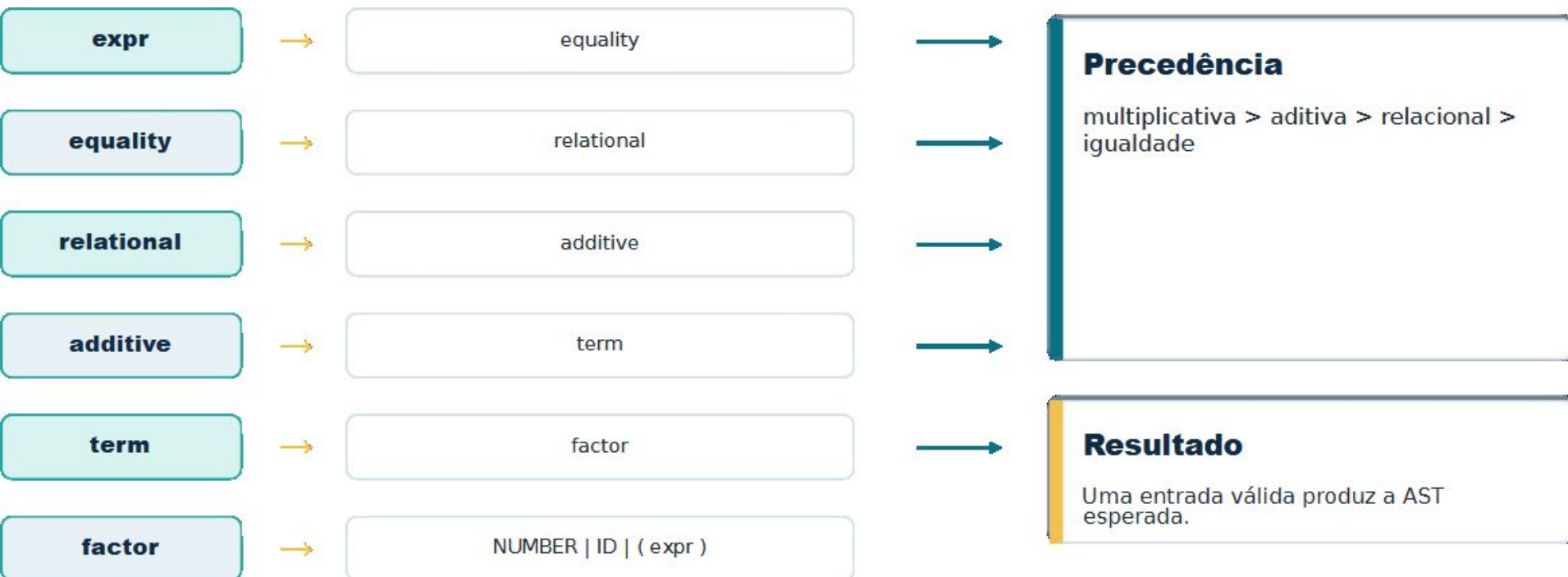
Não-terminal
consulta $M[A,a]$

Se obtivermos \$ e \$
aceita

Caso contrário
erro sintático

Para o MINIC

Separar níveis de precedência torna a estrutura previsível e a AST mais clara.



Para o MINIC

- Gramática documentada e
- Testes de aceitação e rejeição.

01 Separar precedências

lógica, relacional, aditiva, multiplicativa e fator

02 Transformar gramática

eliminar recursão e fatorar alternativas

03 Calcular conjuntos

FIRST e FOLLOW dos não-terminais principais

04 Implementar e testar

consumir entrada, construir AST e diagnosticar erros